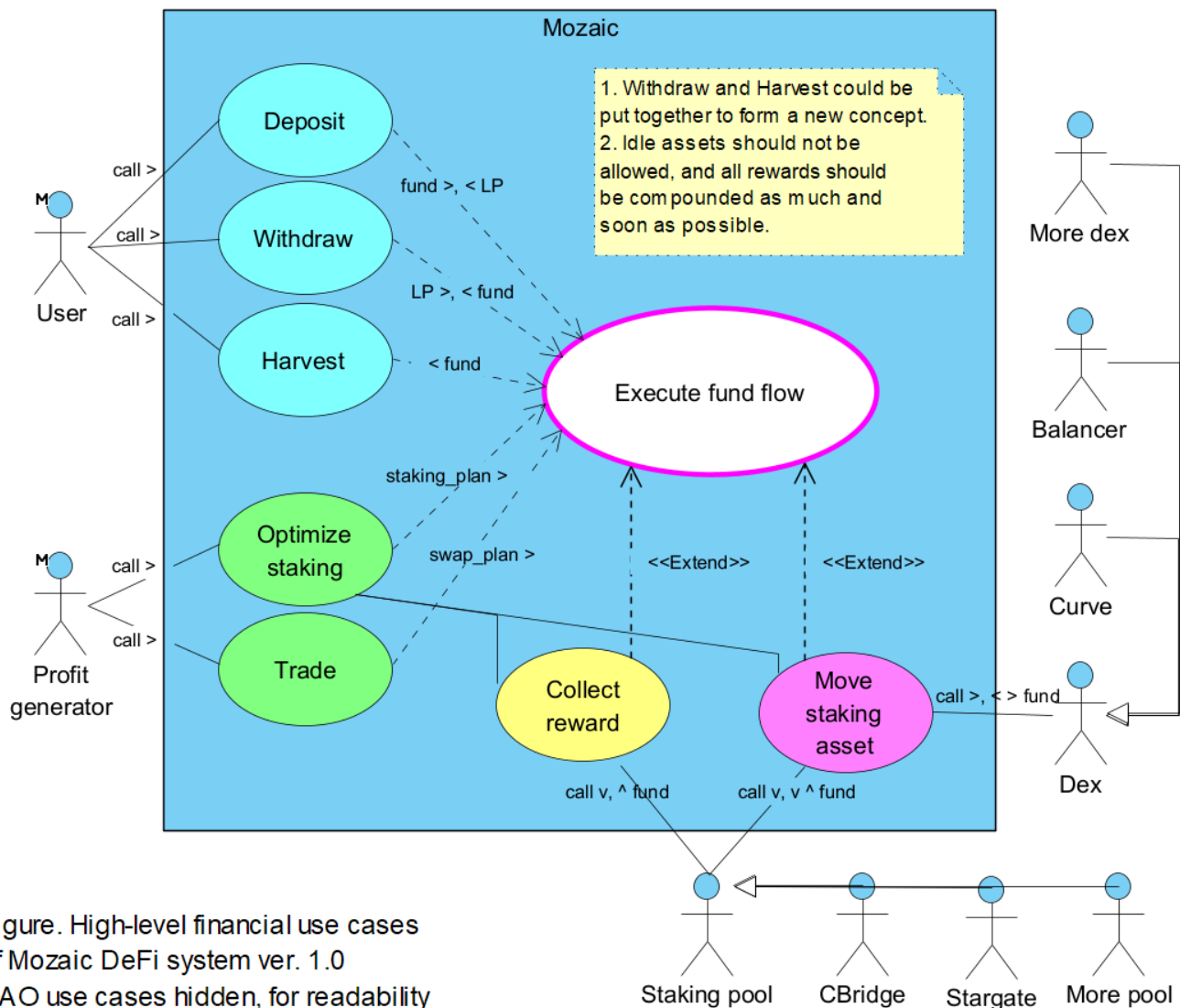


Brainstorming the architecture of asset management

System requirements specified

Mozaic system has aggregational and DAO use cases. Aggregational use cases are shown in the following figure.



The use cases and external actors are described below:

- **Compound:** This hidden use case compounds rewards. Idle assets should not be allowed, and all rewards should be compounded as much and soon as possible.
- **Execute fund flow:** This use case checks, carry out, and keeps track of assets move. The assets managed by the system can only be moved by this use case transparently.
- **Deposit:** This use case deposit the user's assets in the system. **User** calls this use case in the hope that Mozaic system will **Optimize staking** of the deposited assets for them. This use case includes **Execute fund flow**.

- **Withdraw:** This use case withdraws the user's deposited assets. **User** calls this use case. This use case includes **Execute fund flow**.
- **Harvest:** This use case collects reward allocated to the user's deposited asset. **User** calls this use case. It includes **Execute fund flow**.
- **Drawvest:** This is an alternative use case for **Withdraw** and **Harvest** put together. This use case may replace those two use cases if the community prefers it.
- **Optimize staking:** This use case upgrades the staking of deposited assets to maximize rewards. **Profit generator**, a role of the system, calls this use case. This use case includes **Execute fund flow** and **Compound**. *By providing **Execute fund flow** with **staking_plan**, this use case effectively prevents it from being involved with finding optimal staking portfolio.*
- **Trade:** This use case swaps idle assets to get profit by using price changes. It calls **Dex**. **By providing **Profit generator** calls this use case prevents it from being involved with finding optimal trading orders. It includes **Execute funds flow**.*
- **Collect reward:** This use case collects rewards from Staking pools. Use case **Execute fund flow**, when it is working under **Optimize staking**, is extended by this use case. This use case calls Staking pool.
- **Move staking asset:** This use case move assets to/between/from, **Staking pools**. **Execute fund flow**, when it is working under **Optimize staking**, is extended by this use case. This use case calls Staking pool.
- **Dex:** This actor is a smart contract that swaps between assets. Examples are pairs on Curve and Balancer DeFies.
- **Staking pool:** This actor is a smart contract that allocates reward to assets that are staked in it. Examples are farming pools on CBridge and Stargate DeFies.

Identifying vaults through its surrounding modules

As the first implementation step, we identify the module that executes use case ****Execute fund flow****, as it seems to act as the controller and be one of unique features of the system. The design decisions are as illustrated in the following figure:

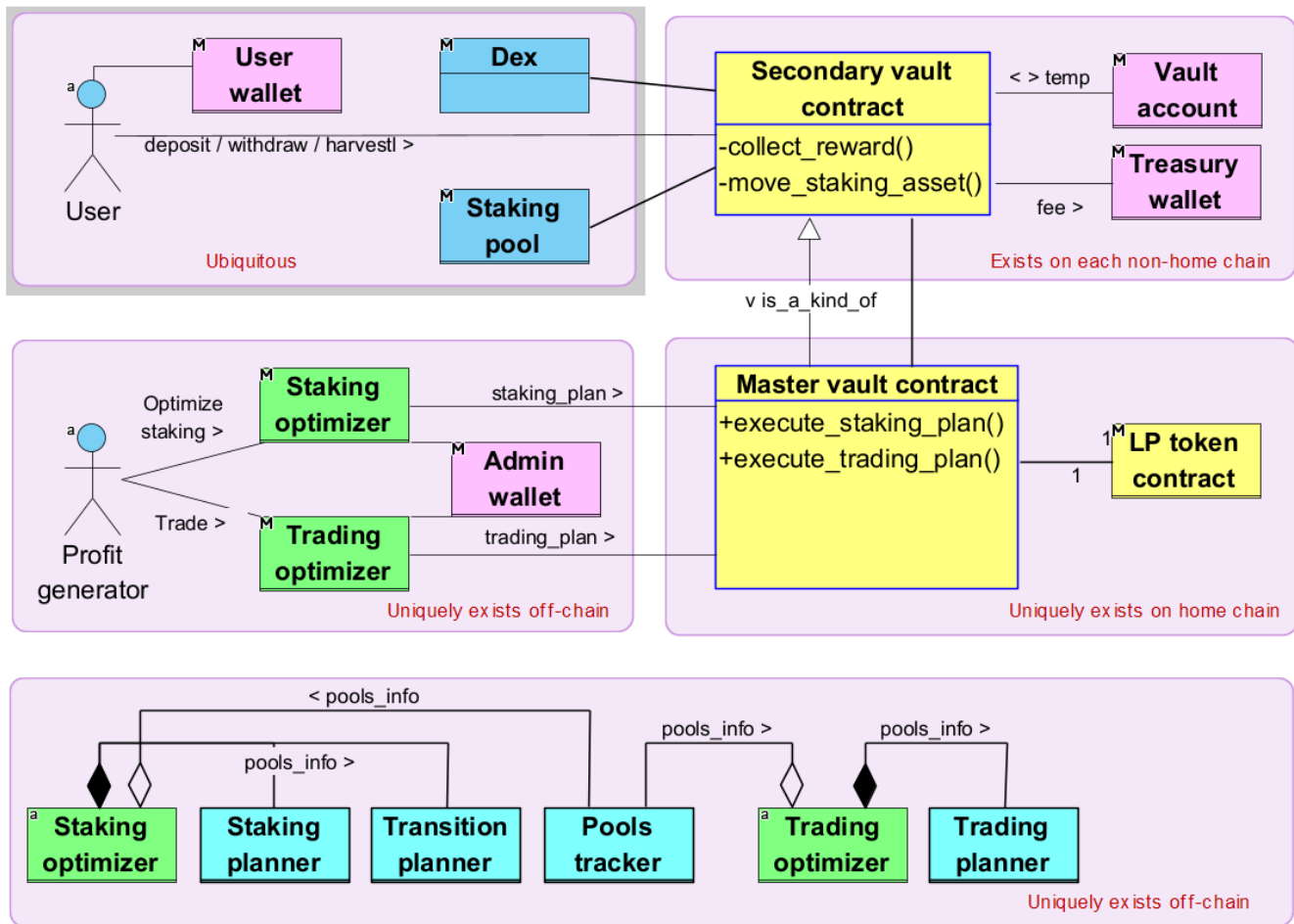


Figure. High-level functional modules of Mozaic DeFi ver. 1.0
Goal: Minimize the functionality of vaults.

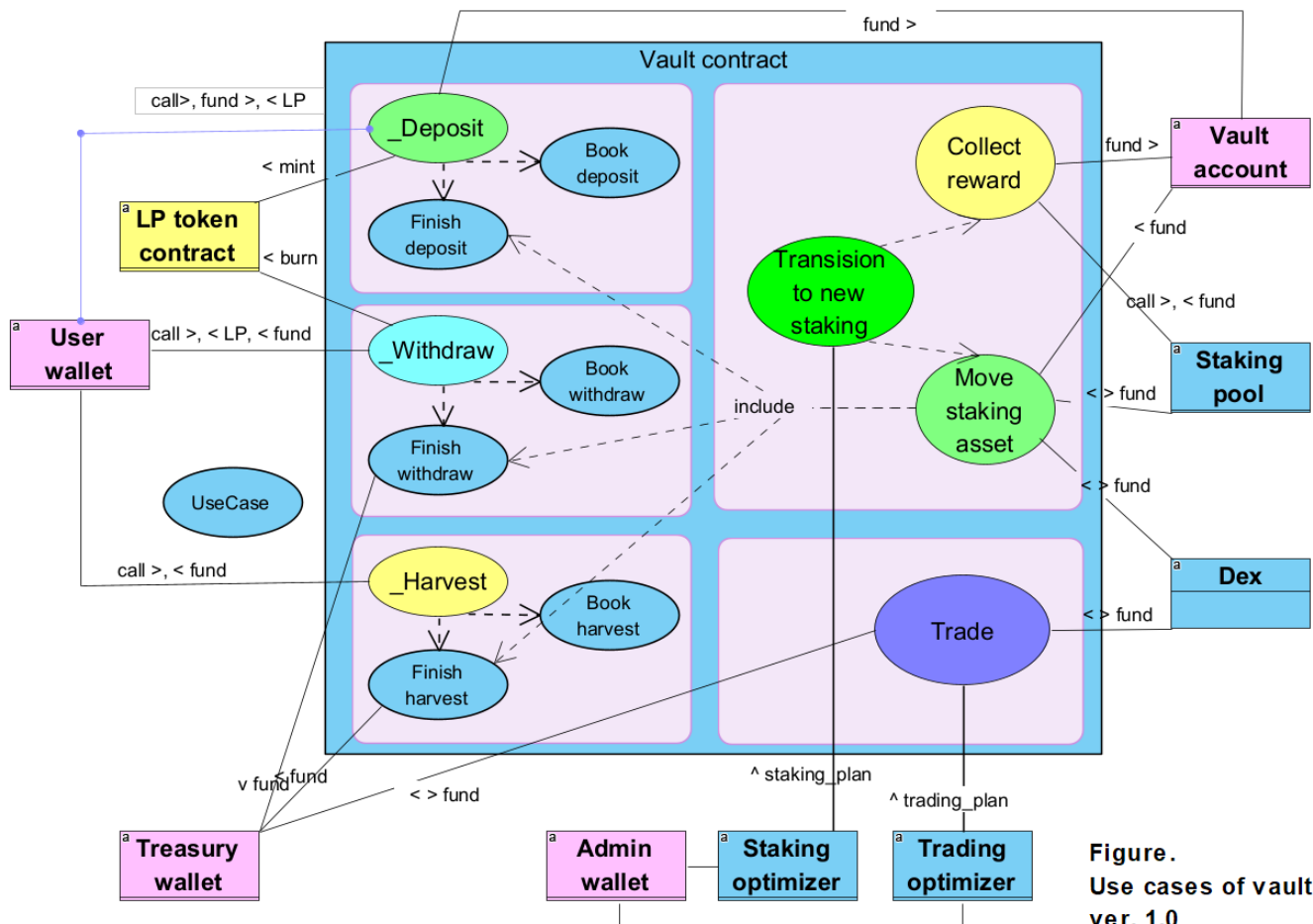
Functional modules are described below:

- **Secondary valut contract:** This module is a smart contract and deployed on each chain except the home chain. It participates in the cooperation with its peer **Secondary vault contracts** to implement **Execute fund flow**, This module has at least the following private operations that work locally on its chain:
 - `collect_reward(...)`
 - `move_staking_asset(...)`
- **Master vault contract:** This module is a smart contract, has global operations, in addition to the local operations inherited from **Secondary vault contract**, and is deployed on one and only one of the chains, called **Home chain**, where it takes the role of **Secondary vault contract**, as well as the the unique role of the master vault operating **LP token contract**.
- **LP token contract:** This module manages the LP token balances of **Users**, which represents the their proportional share of the total assets of the system. Their share comprises of their deposited assets plus automatic compounding. *It is a deliberate design decision that the LP token only exists on **Home chain**.*
- **User wallet:** This is a blockchain wallet and identifies a **User**. **User's** actions; like deposit, withdraw, and harvest; are authenticated/authorized with this wallet.
- **Vault account:** It is the blockchain account of, and controlled by, **Secondary vault contract** and used to store temporary assets, like funds pending staking.

- **Treasury wallet:** This is a blockchain wallet, and a place to store and retrieve system revenues, like fees. It will be better if it is not owned by a human, but be the account of a smart contract that only obeys vault contracts, for better decentralization. It is deployed on all chains.
- **Staking optimizer:** This is an off-chain module that can invoke **Master vault contract**. This module is globally unique, calculates optimal **staking_plans**, and lets the master vault to execute the plans (in cooperation with secondary vaults). *It is an important design decision that the staking_plan is calculated off-chain, thus leading to transparency and security debates, for the sake of gas- and time-savings:*
 - Transparency debate: **Users** will not be able to track why the system chose particular **staking_plans** technically.
 - Security debate: If the calculation of **staking_plan** is hacked or compromised, then the system will make a less-optimal staking maneuver.
 - Justification: Only the second of the following concerns becomes less transparent, leading to both un-assured best profitability and assured huge gas- and time- savings.
 - how much of what assets from which pool to which pool, is the move about
 - whether all the asset moves are securely and/or reasonably/optimally chosen
 - whether all the asset moves are securely executed and logged
 - whether the move logs are readily available to check later
 - whether a staking_plan is executed in an integral and consistent scheme
- **Trading optimizer:** This off-chain module is similar to **Staking optimizer**, except it relates to trading.
- **Admin wallet:** This wallet is used to invoke **"Master vault contract"**, in privilege, on behalf of the administrator.
- **Staking planner:** An integral component of **Staking optimizer**, this module predicts the next most profitable **staking_portfolio**, based on **pools_info** provided by **Pools tracker**. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fee and effectively disable the system.
- **Transition planner:** An integral component of **Staking optimizer**, this module predicts the most efficient **staking_plan**, which is the best procedure of asset move that implements the transitioning to a given **staking_portfolio**, based on the current **pools_info**.
- **Trading optimizer:** This is similar to **Staking optimizer**, except that it relates trading.
- **Trading planner:** This is similar to **Staking planner**, except that it relates trading.
- **Pools tracker:** A shared module between **Staking optimizer** and **Trading optimizer**, this module retrieves and tracks all relevant information from chains, like Reward Release Speed, and total Staked LP of each pool. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fee and effectively disable the system.

Exploring vaults

We have identified vaults through their surrounding modules interacting with them. The external actors in the following use case diagram, together with their interactions with vaults are already described above. We can now explore the use cases of vault.



- **_Deposit**: This is what happens at the level of vault contracts when **Deposit** use case is invoked at the system level. This use case has two disconnected continued parts:
 - ****Book deposit**
- **_Withdraw**: This is what happens at the level of vault contracts when **Withdraw** use case is invoked at the system level.
- **_Harvest**: This is what happens at the level of vault contracts when **Harvest** use case is invoked at the system level.

Staking planner algorithm

Transition planner algorithm